

# Introduction to Data Science and Engineering

## - Basics of Machine Learning



Some materials courtesy of  
Rafael A. Irizarry, and are modified  
from the original version.

Zhenqin (Michael) Wu / 吳楨欽

School of Computing and Data Science  
University of Hong Kong

Slide deck originally created by RB Luo



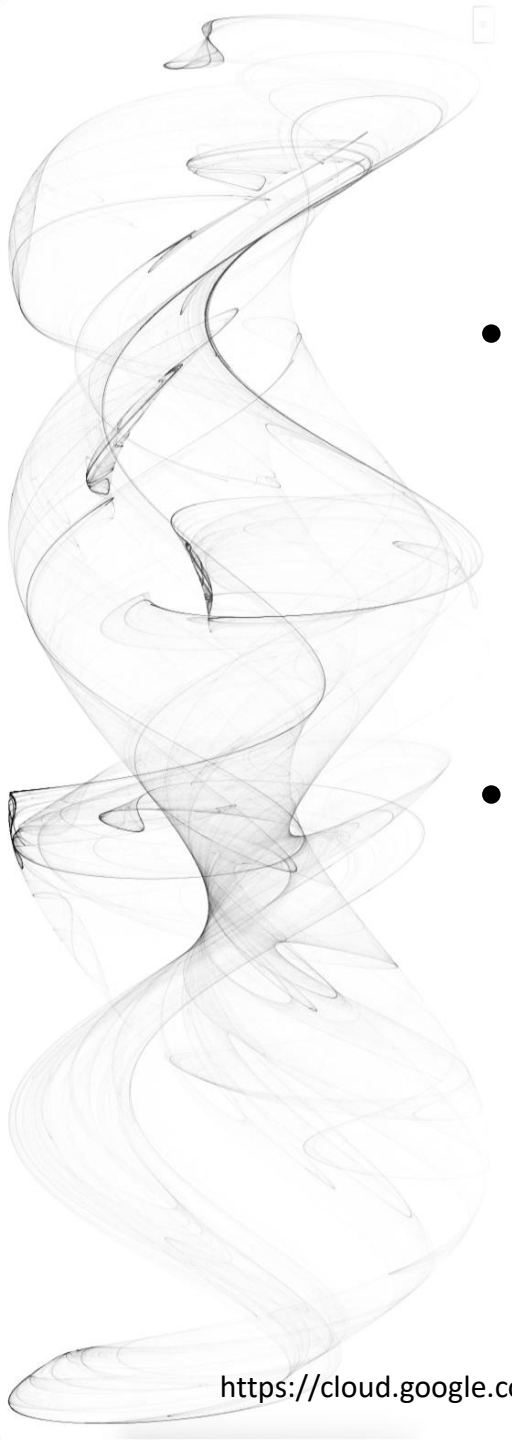
# In this lecture

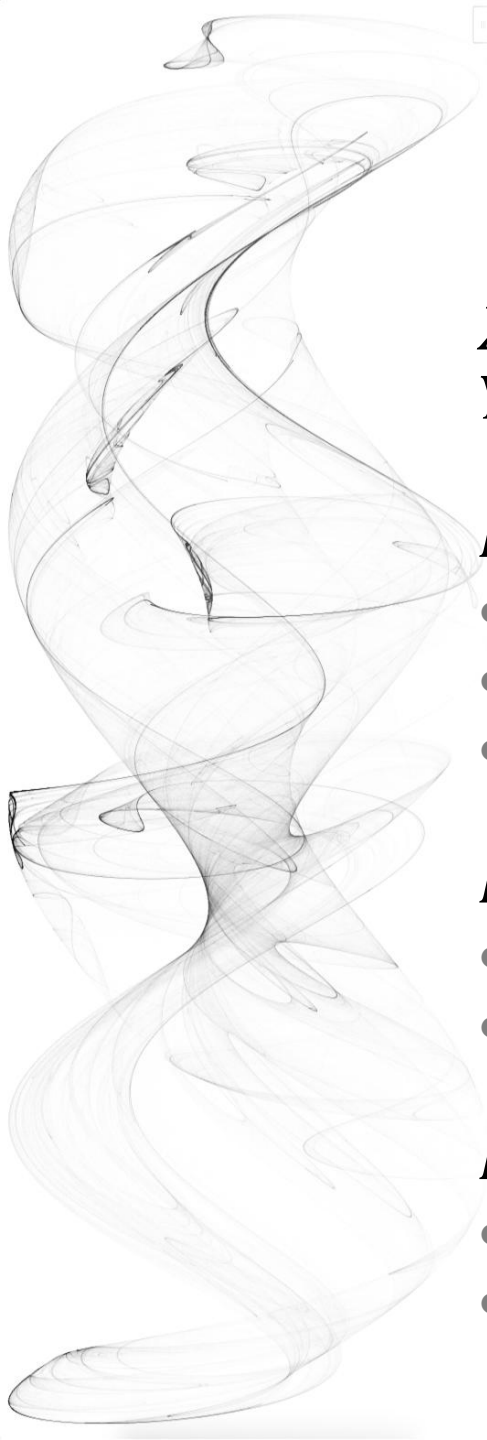
- Definition and notation
- Evaluations
- Decision boundary
- What to use in practice



# In this lecture

- **Definition and notation**
- Evaluations
- Decision boundary
- What to use in practice

- 
- **Artificial intelligence (AI)**
    - The use of technologies to build machines and computers that can mimic cognitive functions associated with human intelligence, such as being able to see, understand, and respond to spoken or written language, analyze data, make recommendations, and more.
  - **Machine learning (ML)**
    - A subset of artificial intelligence that enables a machine or system to learn and improve from existing data.


$$P(X|Y), P(Y|X), P(X)$$

$X$ : observed data, images, texts, features, the cause, etc.

$Y$ : underlying category, characteristics, outcome, the result, etc.

$P(Y|X)$ : Prediction of labels given inputs. Discriminative modeling

- Given email content, is it a spam?
- Given lab results and medical images, diagnose the disease.
- Given a protein sequence, predict its structure

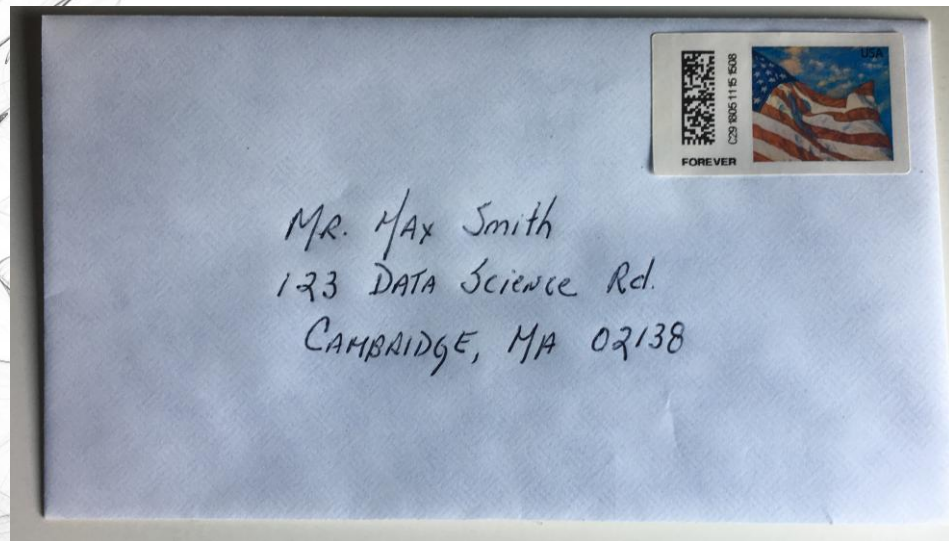
$P(X|Y)$ : Modeling the data distribution given labels. Generative modeling

- Draw an image of a cat
- Generate a molecule that binds to a target protein

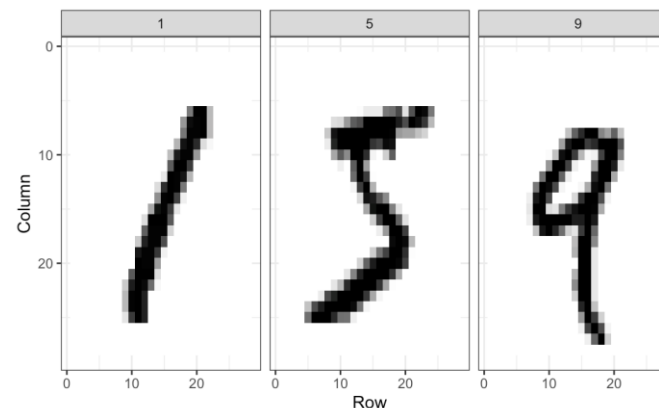
$P(X)$ : Modeling the unconditional data distribution.

- Language modeling, next-word prediction
- Model normal heartbeats and identify irregular ones.

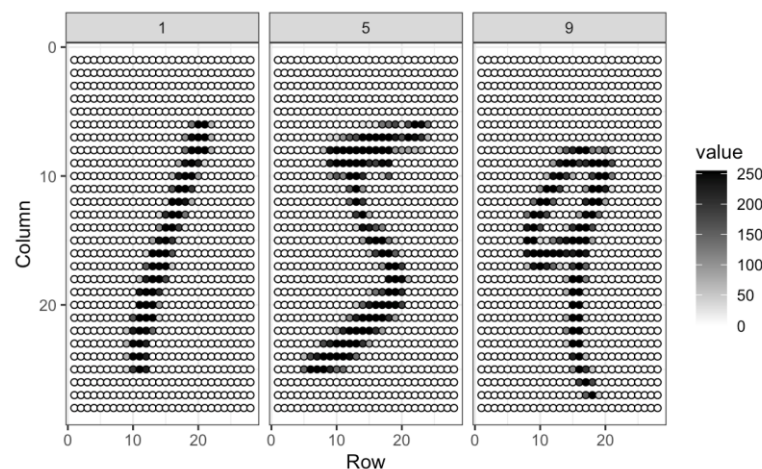
# MNIST: Modified National Institute of Standards and Technology database



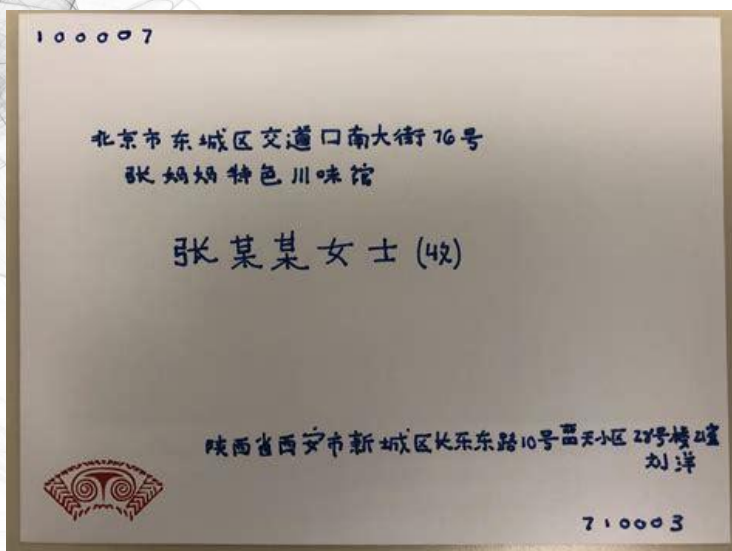
Capture



convert into 28×28 matrix



For each digitized image, we have a categorical label which can be one of 10 values (0,1,2,... , 9)





# More Notation

- $Y$ : the *outcome* we want to predict (the number)
  - When  $Y$  is a categorical variable (i.e., one out of  $K$  classes), this task is called classification
    - Binary classification is the special case when  $K = 2$
  - When  $Y$  is a continuous variable, this task is called regression.
- $X$ : the *features* we use to predict the outcome (the image)
- $P(Y|X)$ : the machine model we will train. This model reads features (images) and predict outcomes (numbers)



# More Notation

When the input has fixed number of features

- $y_i$ : the *outcome* of the  $i$ -th sample.
- $x_{i,j}$ : the  $j$ -th *feature* of the  $i$ -th sample. There are 784 values in a 28x28 matrix.
- $P(y_i|x_i) := f(x_{i,1}, x_{i,2}, x_{i,3}, \dots)$

| outcome  | feature 1 | feature 2 | feature 3 | feature 4 | feature 5 |
|----------|-----------|-----------|-----------|-----------|-----------|
| $y_1$    | $x_{1,1}$ | $x_{1,2}$ | $x_{1,3}$ | $x_{1,4}$ | $x_{1,5}$ |
| $y_2$    | $x_{2,1}$ | $x_{2,2}$ | $x_{2,3}$ | $x_{2,4}$ | $x_{2,5}$ |
| $\vdots$ | $\vdots$  | $\vdots$  | $\vdots$  | $\vdots$  | $\vdots$  |
| $y_n$    | $x_{n,1}$ | $x_{n,2}$ | $x_{n,3}$ | $x_{n,4}$ | $x_{n,5}$ |



# Working on the MNIST example

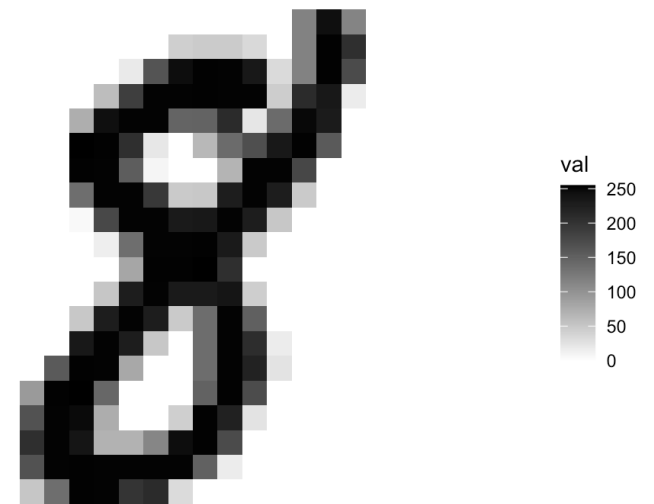
```
library(dslabs)
library(caret)
library(tidyverse)

mnist <- read_mnist()

# Visualize an example image
visualize_sample <- function(img, label, pred=NA){
  img_df <- expand_grid(x=seq(1:28), y=seq(1:28)) |>
    mutate(val = img)
  title <- paste("MNIST digit - label:", label)
  if (!is.na(pred)){
    title <- paste(title, "prediction:", pred)
  }
  img_df |>
    ggplot(aes(x, y, fill = val)) +
    geom_raster() +
    scale_fill_gradient(low = "white", high = "black") +
    scale_y_reverse() + # top-left = (1,1)
    coord_fixed() +
    theme_void() +
    labs(title = title)
}

idx <- 42
img <- mnist$train$images[idx,]
label <- mnist$train$labels[idx]
visualize_sample(img=img, label=label)
```

MNIST digit – label: 8



# Working on the MNIST example

```
# Sample 1,000 training examples
set.seed(123) # For reproducibility
train_index <- sample(nrow(mnist$train$images), 1000)
x_train <- mnist$train$images[train_index, ] / 255.
y_train <- factor(mnist$train$labels[train_index])

# Sample 1,000 test examples
test_index <- sample(nrow(mnist$test$images), 1000)
x_test <- mnist$test$images[test_index, ] / 255.
y_test <- factor(mnist$test$labels[test_index])

# Add column names (required for caret)
colnames(x_train) <- paste0("pxl_", 1:ncol(x_train))
colnames(x_test) <- colnames(x_train)

# Create data frames
train_df <- data.frame(x_train, y = y_train)
test_df <- data.frame(x_test, y = y_test)
```

```
> train_df[1, 750:785]
pxl_750 pxl_751 pxl_752 pxl_753 pxl_754 pxl_755 pxl_756
1      0      0      0      0      0      0      0
pxl_757 pxl_758 pxl_759 pxl_760 pxl_761 pxl_762 pxl_763
1      0      0      0      0      0      0      0
pxl_764 pxl_765 pxl_766 pxl_767 pxl_768 pxl_769 pxl_770
1      0      0      0      0      0      0      0
pxl_771 pxl_772 pxl_773 pxl_774 pxl_775 pxl_776 pxl_777
1      0      0      0      0      0      0      0
pxl_778 pxl_779 pxl_780 pxl_781 pxl_782 pxl_783 pxl_784 y
1      0      0      0      0      0      0      0 6
```

# Working on the MNIST example

```
# Train two models: a KNN model and a XGBoost tree classifier
knn_grid <- expand.grid(k = 5)
knn_model <- train(
  y ~ .,
  method = "knn",
  data = train_df,
  tuneGrid = knn_grid,
  trControl = trainControl(method = "none"))

xgb_grid <- expand.grid(
  nrounds = 100, max_depth = 6, eta = 0.3, gamma = 0,
  colsample_bytree = 0.8, min_child_weight = 1, subsample = 0.8)
xgb_model <- train(
  y ~ .,
  method = "xgbTree",
  data = train_df,
  tuneGrid = xgb_grid,
  trControl = trainControl(method = "none"))
```

```
> xgb_model
eXtreme Gradient Boosting

1000 samples
784 predictor
10 classes: '0', '1', '2', '3', '4', '5', '6', '7', '8', '9'

No pre-processing
Resampling: None
```

```
> y_hat_knn <- predict(knn_model, train_df, type = "raw")
> print(mean((y_hat_knn == y_train)))
[1] 0.933
>
> y_hat_xgb <- predict(xgb_model, train_df, type = "raw")
> print(mean((y_hat_xgb == y_train)))
[1] 1
```

- How can we properly evaluate a model's performance?
- How can we tell which model is better?



# In this lecture

- Definition and notation
- **Evaluations**
- Decision boundary
- What to use in practice



# Evaluations

- Accuracy:  $\frac{\text{Number of exact matches between predictions and labels}}{\text{Total count of samples}}$
- Imagine doing drug efficacy prediction: 1 positive case out of a total of 100 samples.
  - A model picks out 5 most promising candidates, including the real one. Accuracy is 0.96
  - A model predicts negative for everything. Accuracy is 0.99
- Binary classification metrics: precision, recall, F1, ROC-AUC, etc.
  - We will start with an example: **predict gender using height**



# Case study: height => gender

```
> library(tidyverse)
> library(caret)
> library(dslabs)
> data(heights)
```

```
> y <- heights$sex
> x <- heights$height
```

```
> set.seed(42)
> test_index <- createDataPartition(y, times = 1, p = 0.5, list = FALSE)
> test_set <- heights[test_index, ]
> train_set <- heights[-test_index, ]
> train_set |> head(5)
```

|    | sex  | height |
|----|------|--------|
| 1  | Male | 75     |
| 4  | Male | 74     |
| 5  | Male | 61     |
| 10 | Male | 67     |
| 13 | Male | 69     |

- $X$ : height
- $Y$ : gender, Male or Female.
- $P(Y|X)$ : ?

- Train/test split
  - How are we going to use the model?
  - How to simulate that scenario when constructing the model?





# Case study: height => gender

```
> library(tidyverse)
> library(caret)
> library(dslabs)
> data(heights)
```

```
> y <- heights$sex
> x <- heights$height
```

- $X$ : height
- $Y$ : gender, Male or Female.
- $P(Y|X)$ : Let's just do random guess
  - This gives us the random baseline
  - Whatever we do, it should at least be better than guessing

```
> y_hat_test <- sample(c("Male", "Female"), length(test_index), replace=TRUE)
> acc <- mean(y_hat_test == test_set$sex)
> acc
[1] 0.5295238
```

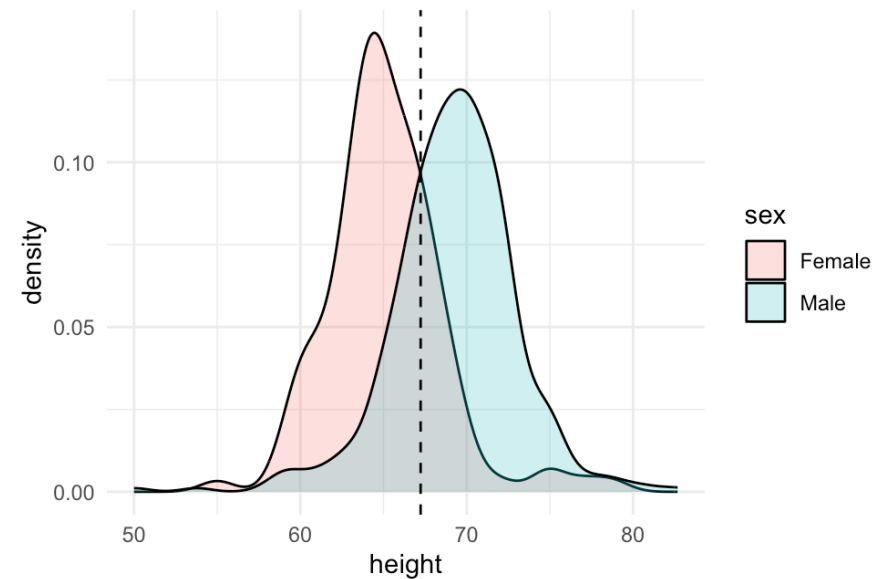
# Case study: height => gender

```
> library(tidyverse)
> library(caret)
> library(dslabs)
> data(heights)
```

```
> y <- heights$sex
> x <- heights$height
```

```
> train_set |>
+   ggplot(aes(height, fill=sex, group=sex)) +
+   geom_density(alpha=0.25) +
+   theme_minimal() +
+   geom_vline(xintercept=67.23, lty="dashed")
```

- $X$ : height
- $Y$ : gender, Male or Female.
- $P(Y|X)$ : thresholding

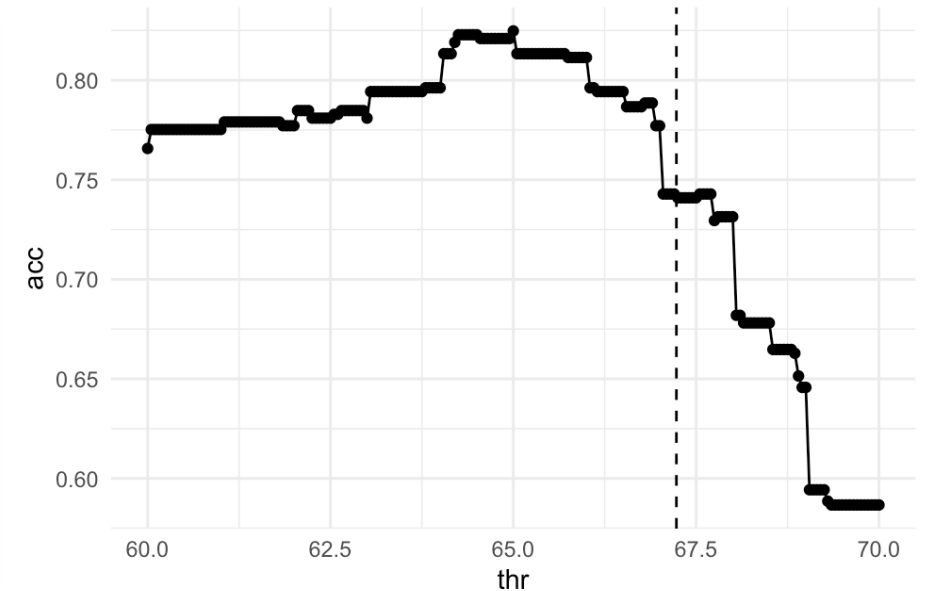


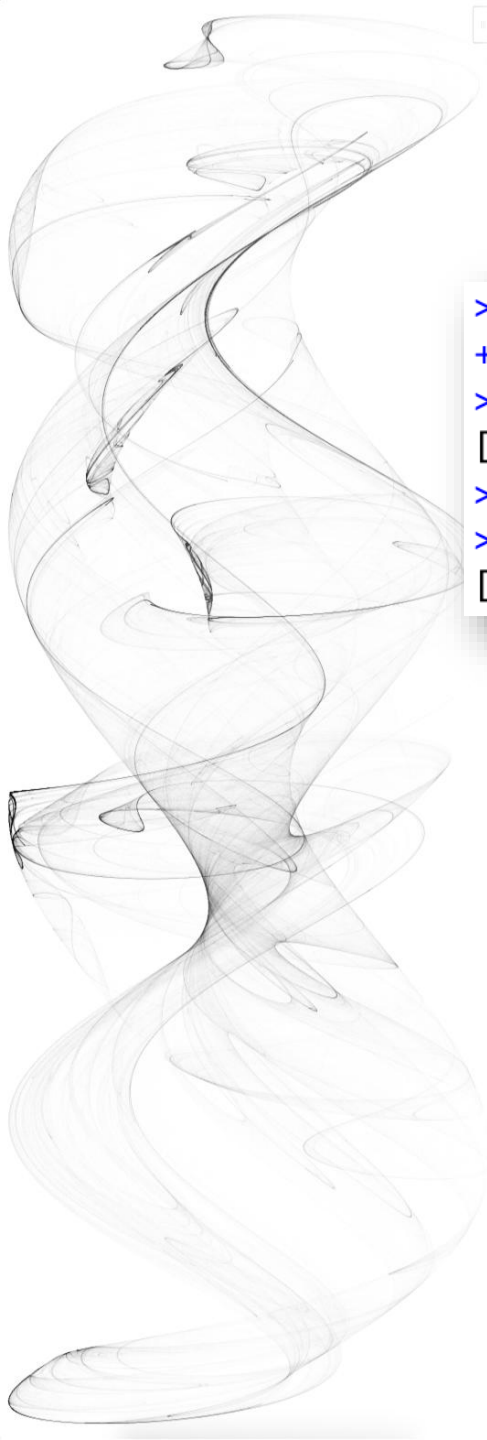


# Case study: height => gender

- $X$ : height
- $Y$ : gender, Male or Female.
- $P(Y|X)$ : thresholding, best at 65 inches => 82.5% accuracy
  - Technically we can also do multiple thresholds

```
predict_by_threshold <- function(threshold){  
  y_hat <- ifelse(train_set$height >= threshold, "Male", "Female")  
  mean(y_hat == train_set$sex)  
}  
  
thresholding_performance <- data.frame(thr=seq(60, 70, 0.05))  
thresholding_performance |>  
  mutate(acc=apply(thr, predict_by_threshold)) |>  
  ggplot(aes(thr, acc)) +  
    geom_point() +  
    geom_line() +  
    geom_vline(xintercept=67.23, lty="dashed") +  
    theme_minimal()
```



A decorative graphic on the left side of the slide consisting of multiple overlapping, wavy, grey lines that create a sense of motion and depth.

# Case study: height => gender

```
> best_threshold <- thresholding_performance |>  
+   arrange(desc(acc)) |> head(1) |> pull(thr)  
> best_threshold  
[1] 65  
> y_test_hat <- ifelse(test_set$height >= best_threshold, "Male", "Female")  
> mean(y_test_hat == test_set$sex)  
[1] 0.8438095
```

- $P(Y|X)$ : this threshold also works for the test data
- What is the best threshold for the test data?
  - We shall never know

A decorative graphic on the left side of the slide consisting of multiple overlapping, wavy, grey lines that create a sense of movement and depth.

# Case study: height => gender

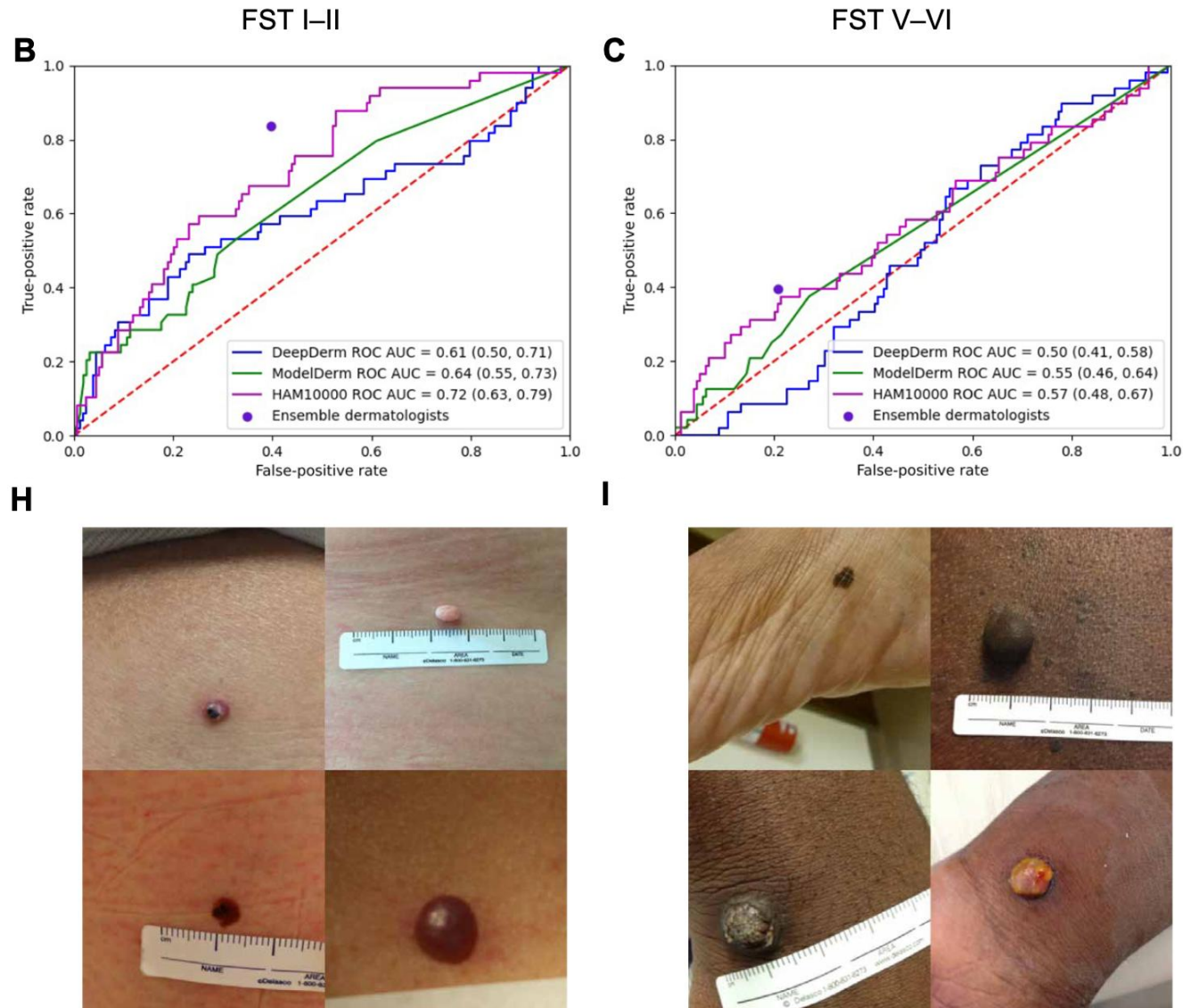
- For the failed predictions, are we mis-classifying females as males? Or the reverse?
- To know more about the distributions of predictions versus ground truths, we examine the **confusion matrix**.

```
> y_hat <- ifelse(train_set$height >= best_threshold, "Male", "Female")  
> table(pred=y_hat, label=train_set$sex)
```

|        | label  |      |
|--------|--------|------|
| pred   | Female | Male |
| Female | 58     | 31   |
| Male   | 61     | 375  |

- We are classifying over half of the females as males when using a threshold that yields highest accuracy. (Why is that?)

# Bias in dermatology AI performance





# More evaluation metrics

Accuracy:  $(TP+TN) / \text{total data points}$

- Among all positives and negatives, how many of them are predicted correctly

Precision:  $TP / (TP + FP)$

- Among all the positive predictions, how many are true.

Recall/Sensitivity:  $TP / (TP + FN)$

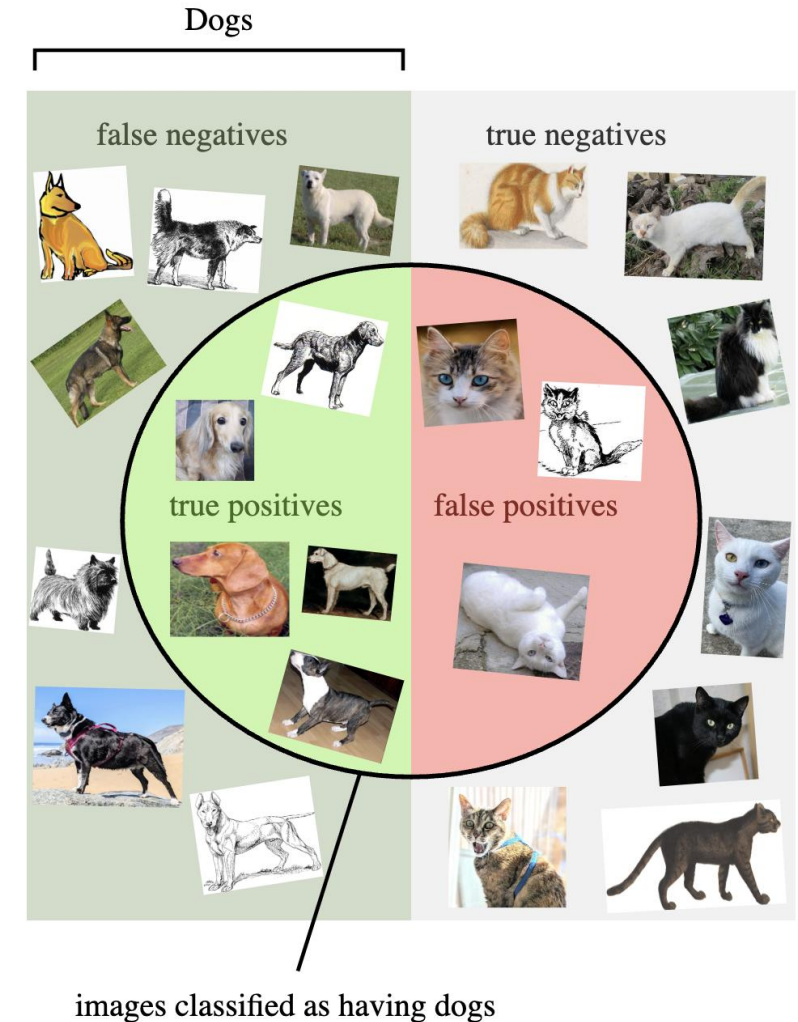
- Among all the actual positives, how many are retrieved

Specificity :  $TN / (TN + FP)$

- Among all the actual negatives, how many are retrieved

[https://en.wikipedia.org/wiki/Precision\\_and\\_recall](https://en.wikipedia.org/wiki/Precision_and_recall)

|                    | Actually Positive    | Actually Negative    |
|--------------------|----------------------|----------------------|
| Predicted positive | True positives (TP)  | False positives (FP) |
| Predicted negative | False negatives (FN) | True negatives (TN)  |



# More evaluation metrics

Sources: [6][7][8][9][10][11][12][13][14] [view](#) · [talk](#) · [edit](#)

|                  |                                                                         | Predicted condition                                                                                                                                |                                                                                          | Sources: <a href="#">[6]</a> <a href="#">[7]</a> <a href="#">[8]</a> <a href="#">[9]</a> <a href="#">[10]</a> <a href="#">[11]</a> <a href="#">[12]</a> <a href="#">[13]</a> <a href="#">[14]</a> <a href="#">view</a> · <a href="#">talk</a> · <a href="#">edit</a> |                                                                                                                              |                                                                                                                   |
|------------------|-------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
|                  |                                                                         | Total population<br>= P + N                                                                                                                        | Positive (PP)                                                                            | Negative (PN)                                                                                                                                                                                                                                                        | Informedness, bookmaker<br>informedness (BM)<br>= TPR + TNR − 1                                                              | Prevalence threshold (PT)<br>$= \frac{\sqrt{\text{TPR} \times \text{FPR}} - \text{FPR}}{\text{TPR} - \text{FPR}}$ |
| Actual condition | Positive (P)                                                            | True positive (TP),<br>hit                                                                                                                         | False negative (FN),<br>type II error, miss,<br>underestimation                          | True positive rate (TPR), recall,<br>sensitivity (SEN),<br>probability of detection, hit rate, power<br>$= \frac{\text{TP}}{\text{P}} = 1 - \text{FNR}$                                                                                                              | False negative rate (FNR),<br>miss rate<br>$= \frac{\text{FN}}{\text{P}} = 1 - \text{TPR}$                                   |                                                                                                                   |
|                  | Negative (N)                                                            | False positive (FP),<br>type I error, false alarm,<br>overestimation                                                                               | True negative (TN),<br>correct rejection                                                 | False positive rate (FPR),<br>probability of false alarm, fall-out<br>$= \frac{\text{FP}}{\text{N}} = 1 - \text{TNR}$                                                                                                                                                | True negative rate (TNR),<br>specificity (SPC), selectivity<br>$= \frac{\text{TN}}{\text{N}} = 1 - \text{FPR}$               |                                                                                                                   |
|                  | Prevalence<br>$= \frac{\text{P}}{\text{P} + \text{N}}$                  | Positive predictive value (PPV),<br>precision<br>$= \frac{\text{TP}}{\text{PP}} = 1 - \text{FDR}$                                                  | False omission rate<br>(FOR)<br>$= \frac{\text{FN}}{\text{PN}} = 1 - \text{NPV}$         | Positive likelihood ratio (LR+)<br>$= \frac{\text{TPR}}{\text{FPR}}$                                                                                                                                                                                                 | Negative likelihood ratio (LR−)<br>$= \frac{\text{FNR}}{\text{TNR}}$                                                         |                                                                                                                   |
|                  | Accuracy (ACC)<br>$= \frac{\text{TP} + \text{TN}}{\text{P} + \text{N}}$ | False discovery rate (FDR)<br>$= \frac{\text{FP}}{\text{PP}} = 1 - \text{PPV}$                                                                     | Negative predictive<br>value (NPV) = $\frac{\text{TN}}{\text{PN}}$<br>$= 1 - \text{FOR}$ | Markedness (MK), deltaP (Δp)<br>$= \text{PPV} + \text{NPV} - 1$                                                                                                                                                                                                      | Diagnostic odds ratio (DOR)<br>$= \frac{\text{LR}_+}{\text{LR}_-}$                                                           |                                                                                                                   |
|                  | Balanced<br>accuracy (BA)<br>$= \frac{\text{TPR} + \text{TNR}}{2}$      | $\text{F}_1 \text{ score} = \frac{2\text{PPV} \times \text{TPR}}{\text{PPV} + \text{TPR}} = \frac{2\text{TP}}{2\text{TP} + \text{FP} + \text{FN}}$ | Fowlkes–Mallows<br>index (FM)<br>$= \sqrt{\text{PPV} \times \text{TPR}}$                 | Matthews correlation coefficient<br>(MCC)<br>$= \sqrt{\text{TPR} \times \text{TNR} \times \text{PPV} \times \text{NPV}}$<br>$- \sqrt{\text{FNR} \times \text{FPR} \times \text{FOR} \times \text{FDR}}$                                                              | Threat score (TS), critical<br>success index (CSI), Jaccard<br>index = $\frac{\text{TP}}{\text{TP} + \text{FN} + \text{FP}}$ |                                                                                                                   |

# More evaluation metrics

```
> confusionMatrix(  
+   data=factor(y_hat, levels=c("Female", "Male")),  
+   reference=train_set$sex)
```

Confusion Matrix and Statistics

|            | Reference |      |
|------------|-----------|------|
| Prediction | Female    | Male |
| Female     | 58        | 31   |
| Male       | 61        | 375  |

Accuracy : 0.8248

95% CI : (0.7895, 0.8563)

No Information Rate : 0.7733

P-Value [Acc > NIR] : 0.002300

Kappa : 0.4512

McNemar's Test P-Value : 0.002499

Sensitivity : 0.4874

Specificity : 0.9236

Pos Pred Value : 0.6517

Neg Pred Value : 0.8601

Prevalence : 0.2267

Detection Rate : 0.1105

Detection Prevalence : 0.1695

Balanced Accuracy : 0.7055

'Positive' Class : Female

- Accuracy is high
- Sensitivity is low (many actually positive points are not correctly retrieved)

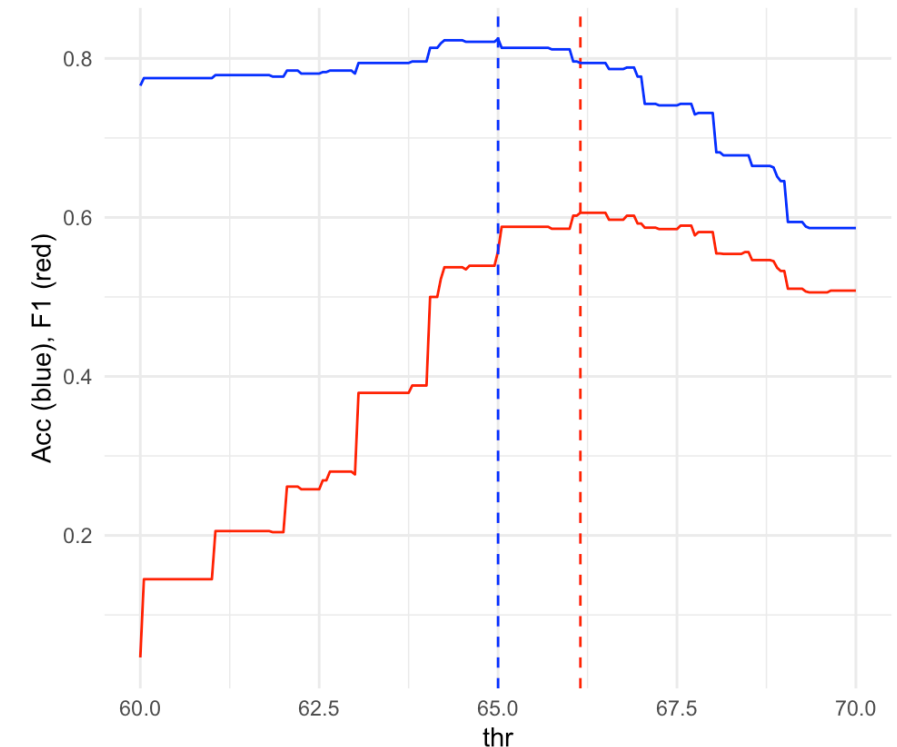


# More evaluation metrics

- F1-score is commonly used in machine learning
- Harmonic mean of precision and recall/sensitivity:

$$F = \frac{2}{\frac{1}{\text{Precision}} + \frac{1}{\text{recall}}}$$

```
predict_by_threshold_f1 <- function(threshold){  
  y_hat <- ifelse(train_set$height >= threshold, "Male", "Female")  
  F_meas(  
    data = factor(y_hat, levels=c("Female", "Male")),  
    reference = train_set$sex)  
}  
  
thresholding_performance <- data.frame(thr=seq(60, 70, 0.05))  
thresholding_performance <- thresholding_performance |>  
  mutate(acc=apply(thr, predict_by_threshold)) |>  
  mutate(f1=apply(thr, predict_by_threshold_f1))  
  
thresholding_performance |>  
  ggplot() +  
  geom_line(aes(thr, f1), color="red") +  
  geom_line(aes(thr, acc), color="blue") +  
  ylab("Acc (blue), F1 (red)") +  
  theme_minimal()
```



# More evaluation metrics

- Use F1 score to select the best threshold
  - Threshold: 65 => 66.15
  - Acc: 0.82 => 0.79
  - Recall/Sensitivity: 0.49 => 0.70

```
> y_hat <- ifelse(train_set$height >= 66.15, "Male", "Female")  
> confusionMatrix(data=factor(y_hat), reference=train_set$sex)
```

Confusion Matrix and Statistics

|            | Reference |      |
|------------|-----------|------|
| Prediction | Female    | Male |
| Female     | 83        | 72   |
| Male       | 36        | 334  |

Accuracy : 0.7943

95% CI : (0.7571, 0.8281)

No Information Rate : 0.7733

P-Value [Acc > NIR] : 0.1363957

Kappa : 0.4699

Mcnemar's Test P-Value : 0.0007575

Sensitivity : 0.6975

Specificity : 0.8227

Pos Pred Value : 0.5355

Neg Pred Value : 0.9027

Prevalence : 0.2267

Detection Rate : 0.1581

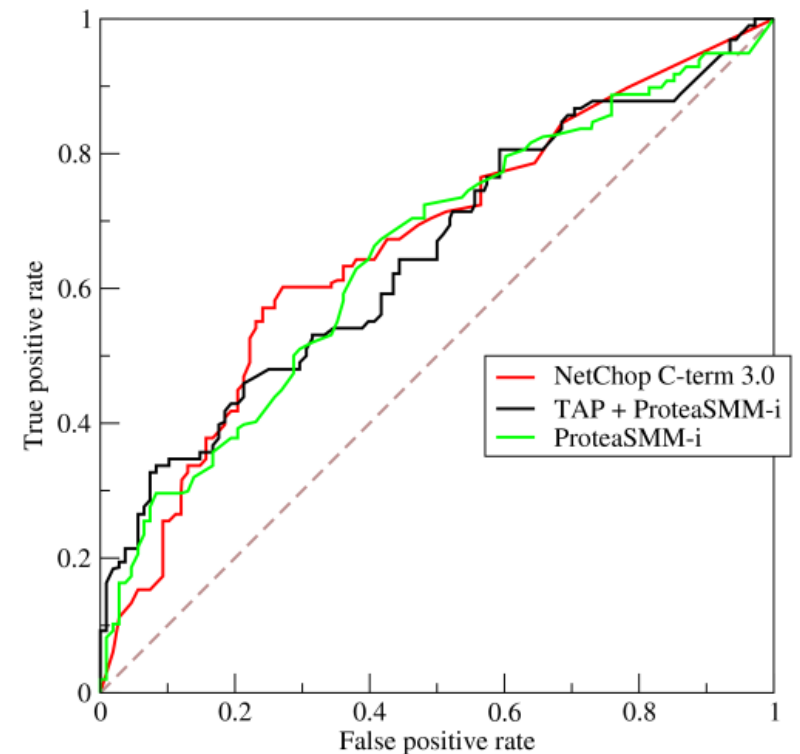
Detection Prevalence : 0.2952

Balanced Accuracy : 0.7601

'Positive' Class : Female

# More evaluation metrics

- What if we don't want to select a threshold?
  - Evaluate the overall correlation between our predictor (height) and label (gender)
- A widely-adopted approach is the receiver operating characteristic (ROC):
  - True positive rate (TPR) / recall / sensitivity versus False positive rate (FPR) / 1 - specificity
  - Derive TPR(s) and FPR(s) at different thresholds;
  - Draw a continuous curve connecting all (TPR, FPR) pairs;
  - Area under the ROC curve (AUROC, or ROC-AUC) as an objective, threshold-agnostic measurement of prediction performance;
  - ROC-AUC = 0.5 as a universal random baseline.



# More evaluation metrics

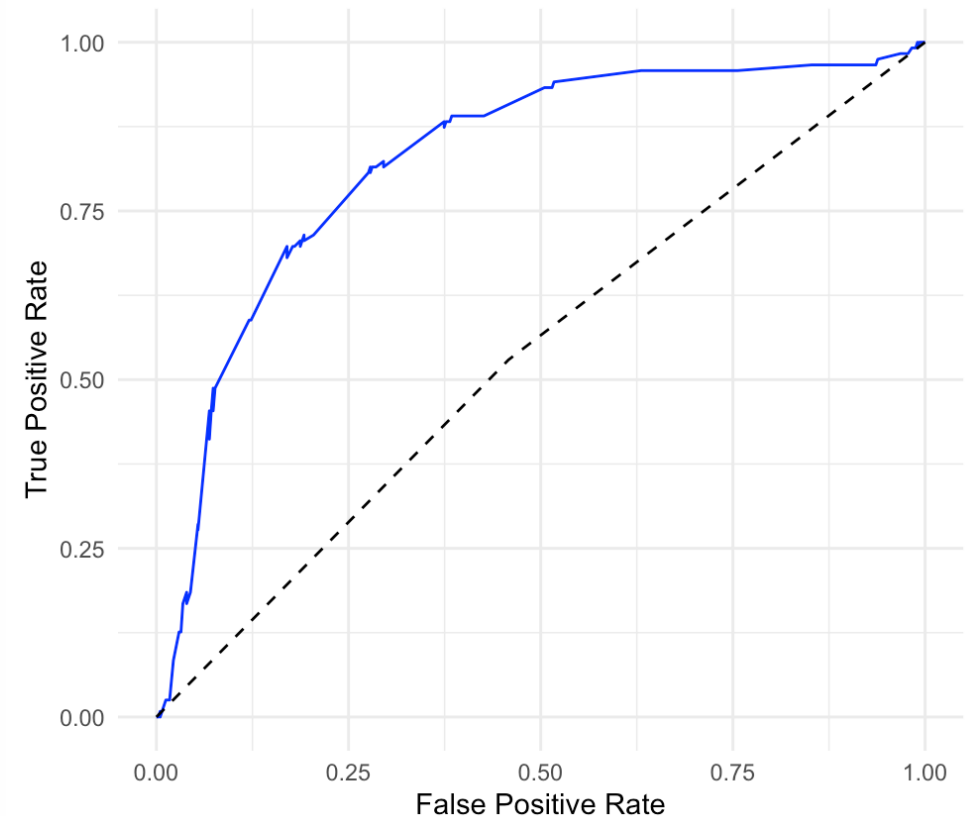
```
## ROC-AUC
labels <- ifelse(train_set$sex == "Female", 1, 0)
preds <- train_set$height
roc_curve <- roc(labels, preds)
random_roc_curve <- roc(labels, sample(c(0, 1), length(labels), replace=TRUE))

roc_df <- data.frame(
  thresholds = roc_curve$thresholds,
  tprs = roc_curve$sensitivities,
  fprs = 1 - roc_curve$specificities)

r_roc_df <- data.frame(
  thresholds = random_roc_curve$thresholds,
  r_tprs = random_roc_curve$sensitivities,
  r_fprs = 1 - random_roc_curve$specificities
)

ggplot() +
  geom_line(aes(fprs, tprs), data=roc_df, color="blue") +
  geom_line(aes(r_fprs, r_tprs), data=r_roc_df, color="black", lty="dashed") +
  xlab("False Positive Rate") +
  ylab("True Positive Rate") +
  theme_minimal()
```

```
> print(auc(random_roc_curve))
Area under the curve: 0.5356
> print(auc(roc_curve))
Area under the curve: 0.8248
```



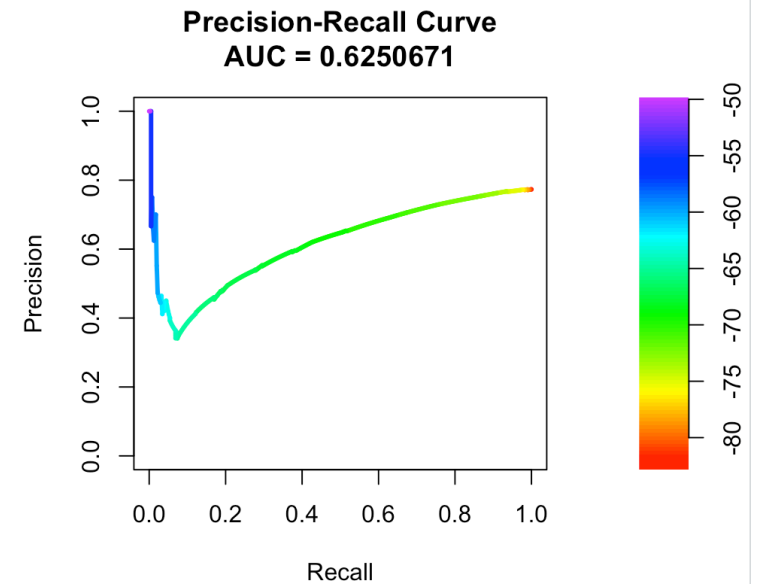
# More evaluation metrics

- What if we don't want to select a threshold?
  - Evaluate the overall correlation between our predictor (height) and label (gender)
- Another alternative: area under the precision-recall curve (auPRC)
  - More effective when the positive class is extremely rare
  - No fixed random baseline value.

```
## au-PRC
library(PRRROC)
pr <- pr.curve(scores.class0 = -train_set$height[train_set$sex == "Male"],
               scores.class1 = -train_set$height[train_set$sex == "Female"],
               curve = TRUE)

# Print AUC for PRC
cat("AUC-PRC:", pr$auc.integral, "\n")

# Plot the Precision-Recall Curve
plot(pr, main = "Precision-Recall Curve")
```

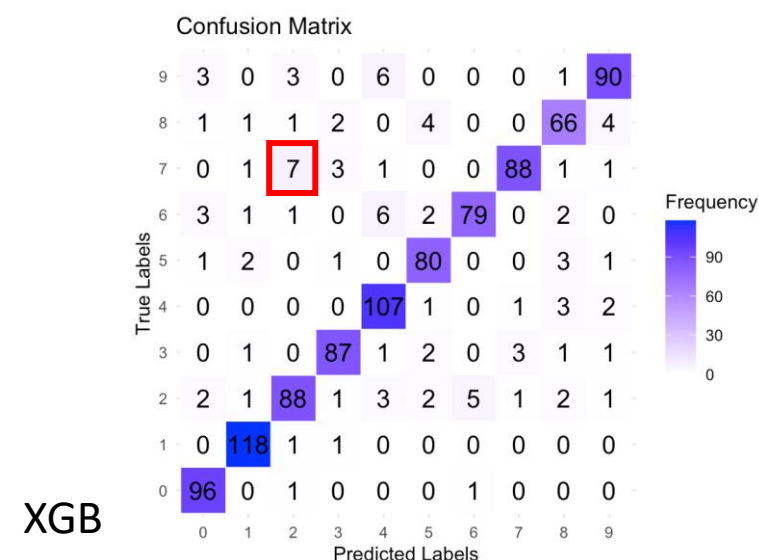
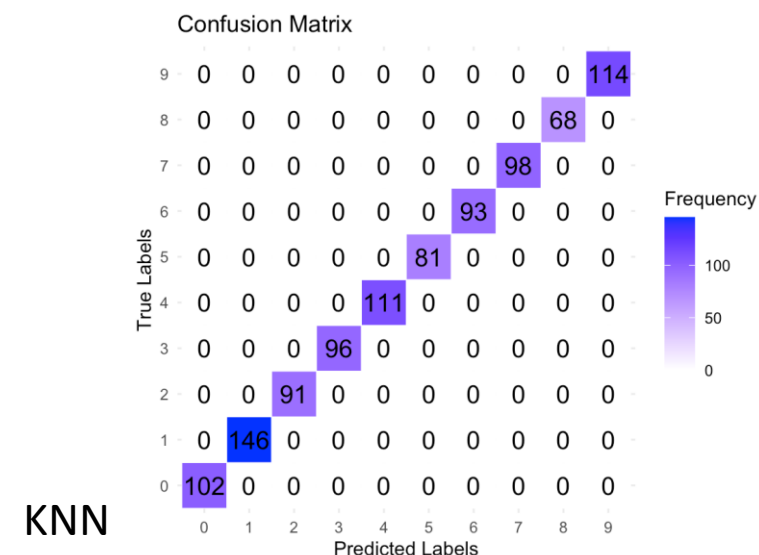




# MNIST: Evaluating multi-class classification

```
plot_conf_mat <- function(labels, preds){  
  confusion_matrix <- table(labels, preds)  
  confusion_df <- as.data.frame(as.table(confusion_matrix))  
  colnames(confusion_df) <- c("True", "Predicted", "Frequency")  
  # Plot the confusion matrix using a heatmap  
  ggplot(confusion_df, aes(x = Predicted, y = True, fill = Frequency)) +  
    geom_tile(color = "white") +  
    geom_text(aes(label = Frequency), color = "black", size = 5) +  
    scale_fill_gradient(low = "white", high = "blue") +  
    labs(  
      title = "Confusion Matrix",  
      x = "Predicted Labels",  
      y = "True Labels"  
    ) +  
    theme_minimal()  
}
```

```
> y_hat_knn <- predict(knn_model, test_df, type = "raw")  
> print(paste("Accuracy", mean(y_hat_knn == test_df$y)))  
[1] "Accuracy 0.895"  
> plot_conf_mat(test_df$y, y_hat_knn)  
> y_hat_xgb <- predict(xgb_model, test_df, type = "raw")  
> print(paste("Accuracy", mean(y_hat_xgb == test_df$y)))  
[1] "Accuracy 0.899"  
> plot_conf_mat(test_df$y, y_hat_xgb)
```







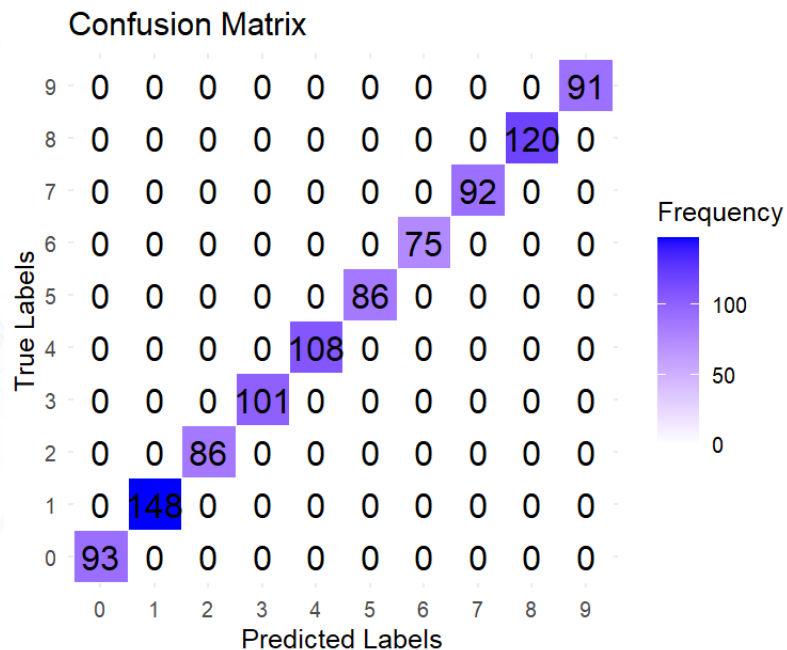
# MNIST: Evaluating multi-class classification

- Overall accuracy
- Per-class Precision / Recall / F1
- Macro / Micro / Weighted-averaged Precision / Recall / F1
  - Macro: same weights to all classes
  - Weighted: weights proportional to class abundance
- One-vs-rest ROC-AUC
- Confusion matrix

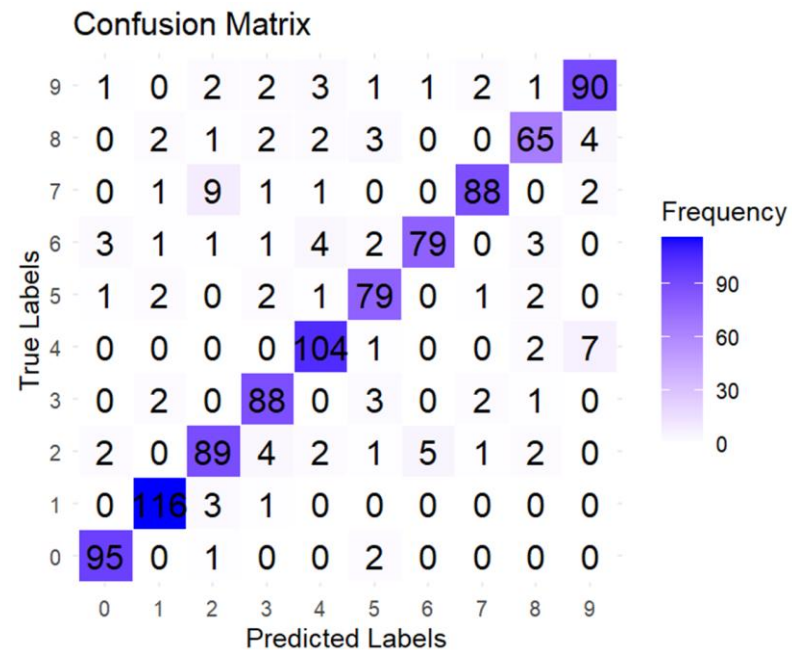
# Overfitting and underfitting

```
> y_hat_xgb_train <- predict(xgb_model, train_df, type = "raw")
> print(paste("Train set accuracy:", mean(y_hat_xgb_train == train_df$y)))
[1] "Train set accuracy: 1"
> plot_conf_mat(train_df$y, y_hat_xgb_train)
> y_hat_xgb <- predict(xgb_model, test_df, type = "raw")
> print(paste("Test set accuracy:", mean(y_hat_xgb == test_df$y)))
[1] "Test set accuracy: 0.893"
> plot_conf_mat(test_df$y, y_hat_xgb)
```

Train set



Test set



# Overfitting and underfitting

```
## Underfitting
set.seed(123)
train_index <- sample(nrow(mnist$train$images), 1000)
x_train <- mnist$train$images[train_index, ] / 255.
y_train <- factor(mnist$train$labels[train_index])
uf_train_df <- data.frame(x_train[,1:150], y = y_train)

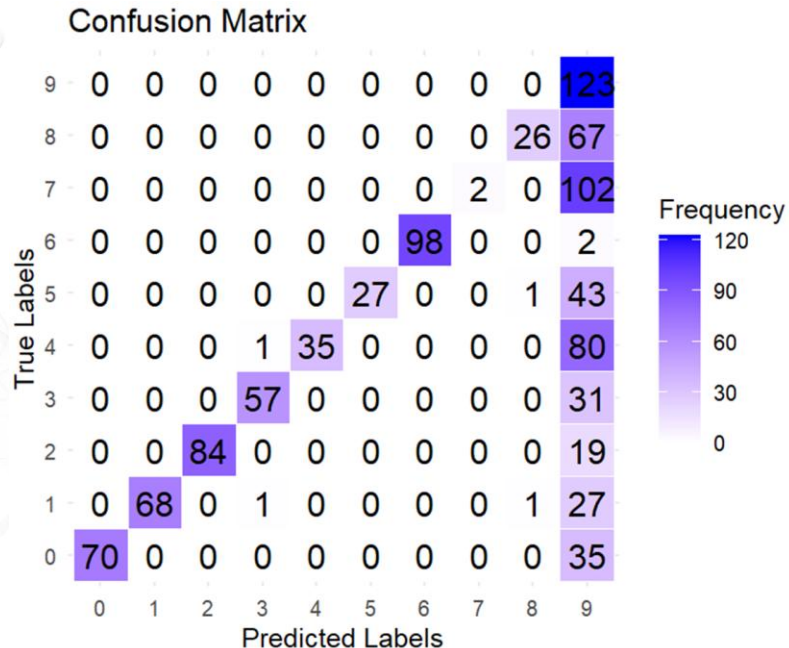
test_index <- sample(nrow(mnist$test$images), 1000)
x_test <- mnist$test$images[test_index, ] / 255.
y_test <- factor(mnist$test$labels[test_index])
uf_test_df <- data.frame(x_test[,1:150], y = y_test)

xgb_grid <- expand.grid(
  nrounds = 100, max_depth = 6, eta = 0.3, gamma = 0,
  colsample_bytree = 0.8, min_child_weight = 1, subsample = 0.8)
uf_xgb_model <- train(
  y ~ .,
  method = "xgbTree",
  data = uf_train_df,
  tuneGrid = xgb_grid,
  trControl = trainControl(method = "none"))
```

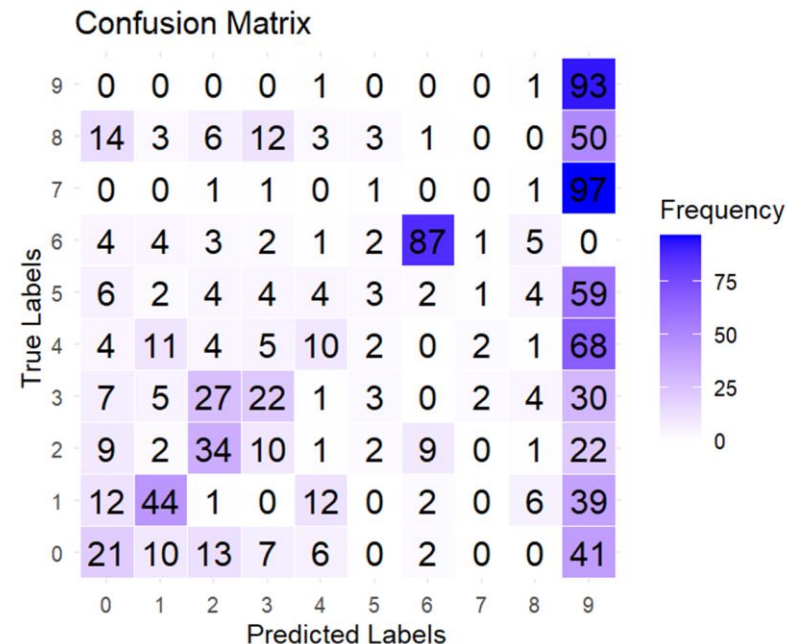
# Overfitting and underfitting

```
> y_hat_xgb_train <- predict(uf_xgb_model, uf_train_df, type = "raw")
> print(paste("Train set accuracy:", mean(y_hat_xgb_train == uf_train_df$y)))
[1] "Train set accuracy: 0.59"
> plot_conf_mat(uf_train_df$y, y_hat_xgb_train)
> y_hat_xgb <- predict(uf_xgb_model, uf_test_df, type = "raw")
> print(paste("Test set accuracy:", mean(y_hat_xgb == uf_test_df$y)))
[1] "Test set accuracy: 0.314"
> plot_conf_mat(uf_test_df$y, y_hat_xgb)
```

Train set



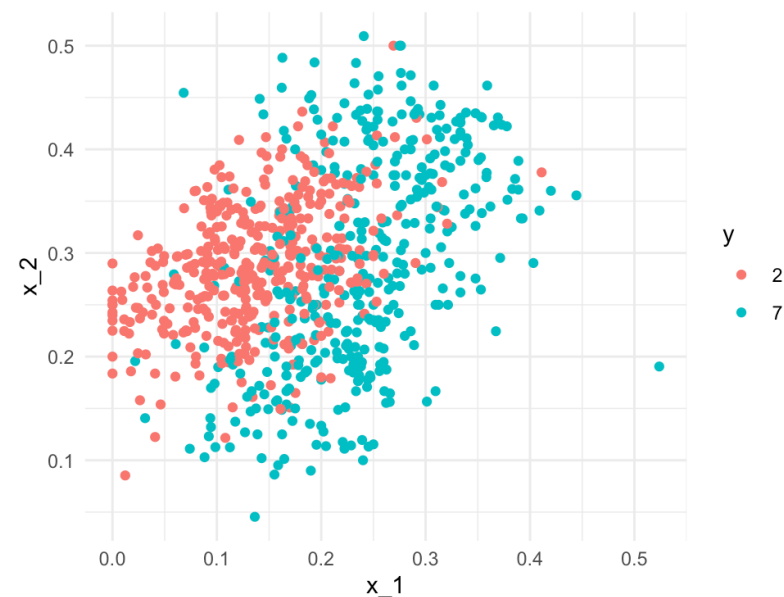
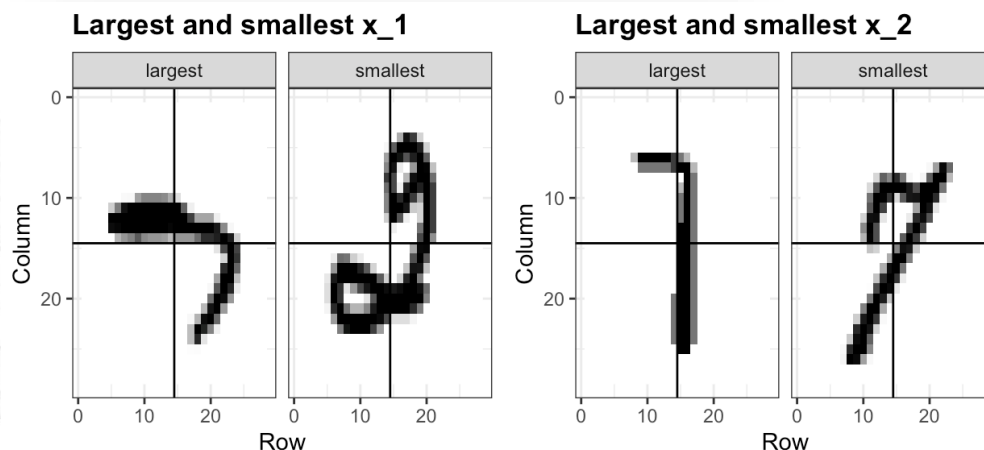
Test set



# Decision boundary

- A simplified version of MNIST: telling if the digit is a 2 or 7
- Features: the proportion of dark pixels that are in the upper left quadrant ( $x_1$ ) and the lower right quadrant ( $x_2$ )

```
data("mnist_27")
mnist_27$train |>
  ggplot(aes(x_1, x_2, color = y)) +
  geom_point() +
  theme_minimal()
```





# Decision boundary

```
> fit <- mnist_27$train |>
+   mutate(y = ifelse(y==7, 1, 0)) |>
+   lm(y ~ x_1 + x_2, data = _)
> p_hat <- predict(fit, newdata = mnist_27$test)
> y_hat <- factor(ifelse(p_hat > 0.5, 7, 2))
> print(paste("Accuracy:", mean(y_hat == mnist_27$test$y)))
[1] "Accuracy: 0.775"
> knn_grid <- expand.grid(k = 5)
> knn_model <- train(
+   y ~ .,
+   method = "knn",
+   data = mnist_27$train,
+   tuneGrid = knn_grid,
+   trControl = trainControl(method = "none"))
> p_hat <- predict(knn_model, newdata = mnist_27$test, type="prob")["7"]
> y_hat <- factor(ifelse(p_hat > 0.5, 7, 2))
> print(paste("Accuracy:", mean(y_hat == mnist_27$test$y)))
[1] "Accuracy: 0.815"
```

0.815 > 0.775  
Why is kNN  
better than  
linear?



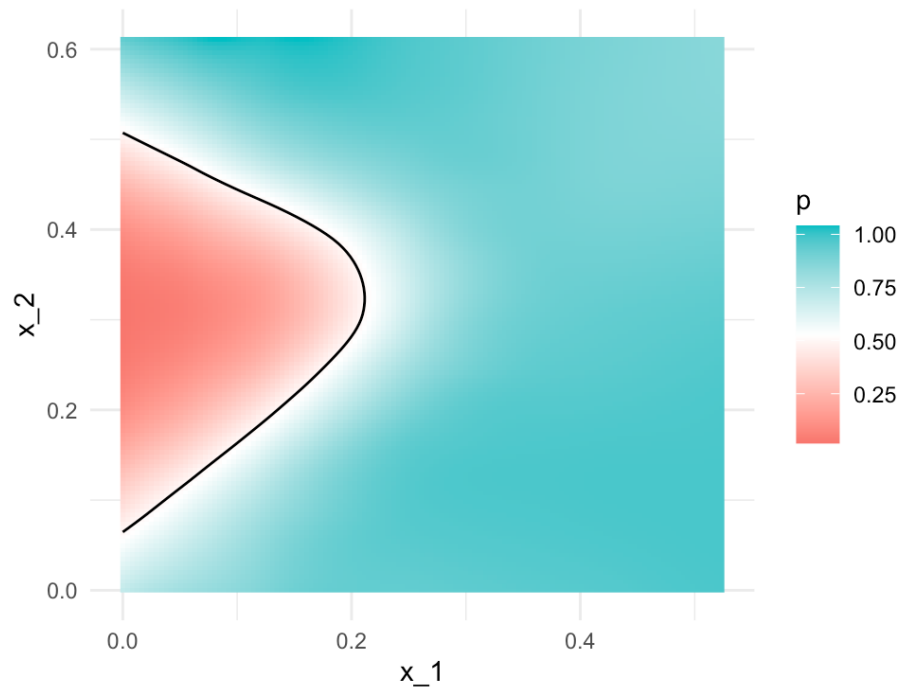


# In this lecture

- Definition and notation
- Evaluations
- **Decision boundary**
- What to use in practice

# Decision boundary

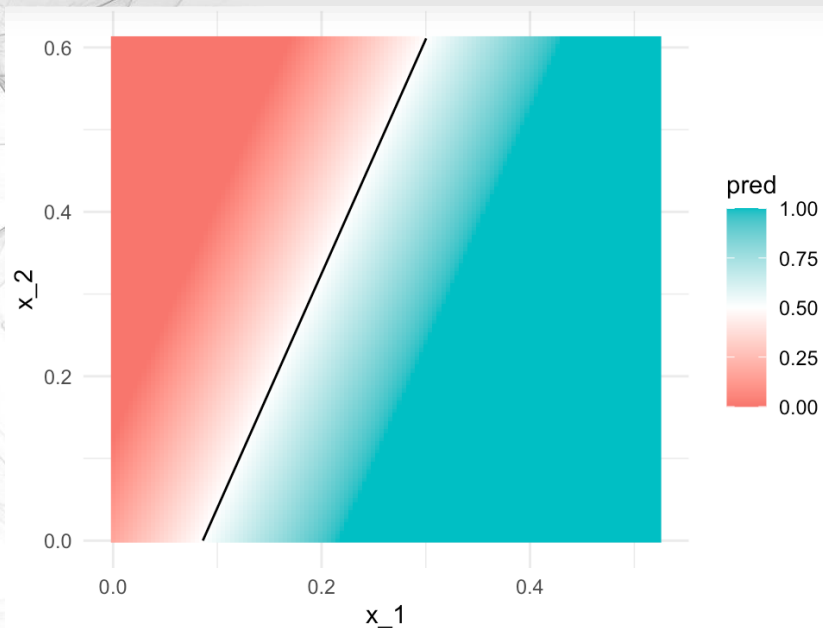
```
mnist_27$true_p |> ggplot(aes(x_1, x_2, z=p, fill=p)) +  
  geom_raster() +  
  scale_fill_gradientn(colors=c("#F8766D", "white", "#00BFC4")) +  
  stat_contour(breaks=c(0.5), color="black") +  
  theme_minimal()
```



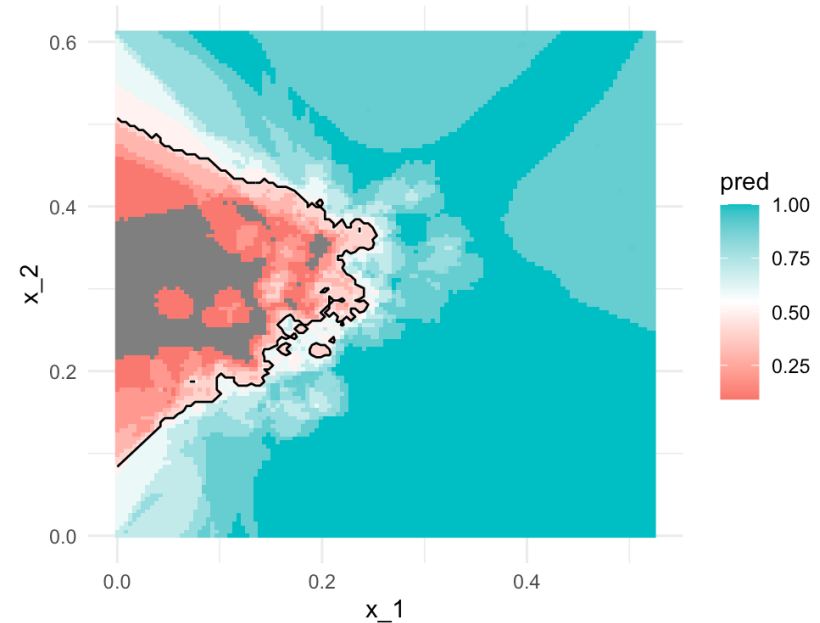
The (theoretically)  
optimal decision boundary

# Decision boundary

```
grid_input <- mnist_27$true_p |> select(x_1, x_2)
grid_input |>
  mutate(pred = predict(fit, newdata = grid_input)) |>
  mutate(pred = pmax(0, pmin(pred, 1))) |>
  ggplot(aes(x_1, x_2, z=pred, fill=pred)) +
  geom_raster() +
  scale_fill_gradientn(
    colors=c("#F8766D", "white", "#00BFC4"),
    limits = c(0, 1)) +
  stat_contour(breaks=c(0.5), color="black") +
  theme_minimal()
```

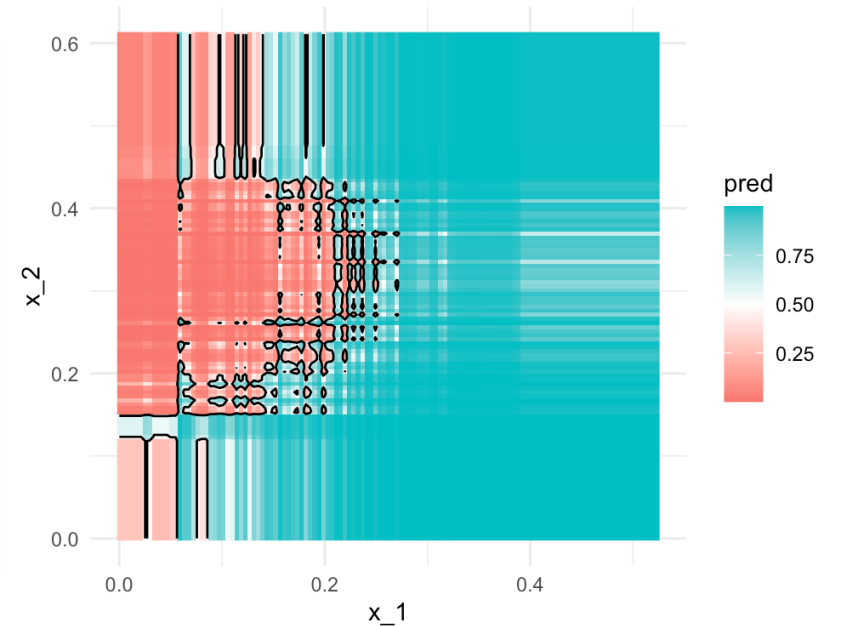


```
grid_input |>
  mutate(pred = predict(knn_model, newdata = grid_input, type = "prob")["7"]) |>
  mutate(pred = pmax(0, pmin(pred, 1))) |>
  ggplot(aes(x_1, x_2, z=pred, fill=pred)) +
  geom_raster() +
  scale_fill_gradientn(
    colors=c("#F8766D", "white", "#00BFC4")) +
  stat_contour(breaks=c(0.5), color="black") +
  theme_minimal()
```



# Decision boundary

```
> xgb_grid <- expand.grid(
+   nrounds = 100, max_depth = 6, eta = 0.3, gamma = 0,
+   colsample_bytree = 0.8, min_child_weight = 1, subsample = 0.8)
> xgb_model <- train(
+   y ~ .,
+   method = "xgbTree",
+   data = mnist_27$train,
+   tuneGrid = xgb_grid,
+   trControl = trainControl(method = "none"))
> p_hat <- predict(xgb_model, newdata = mnist_27$test, type="prob")["7"]
> y_hat <- factor(ifelse(p_hat > 0.5, 7, 2))
> print(paste("Accuracy:", mean(y_hat == mnist_27$test$y)))
[1] "Accuracy: 0.825"
```





# In this lecture

- Definition and notation
- Evaluations
- Decision boundary
- **What to use in practice**



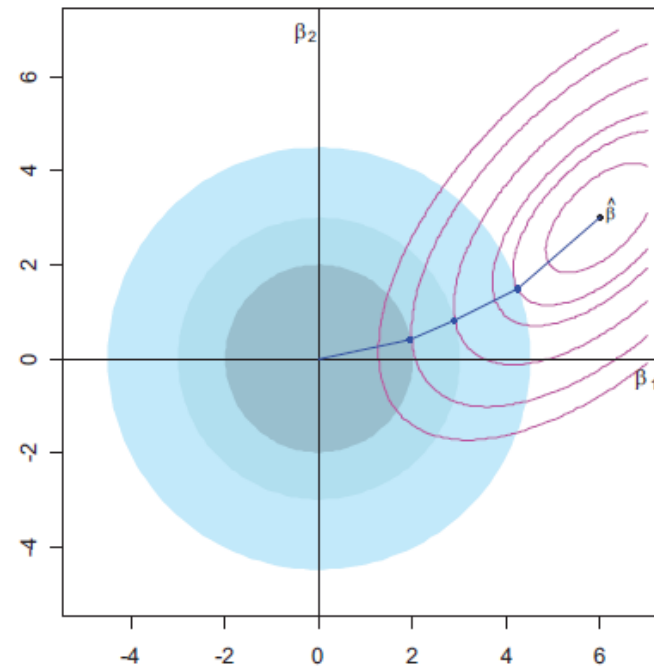
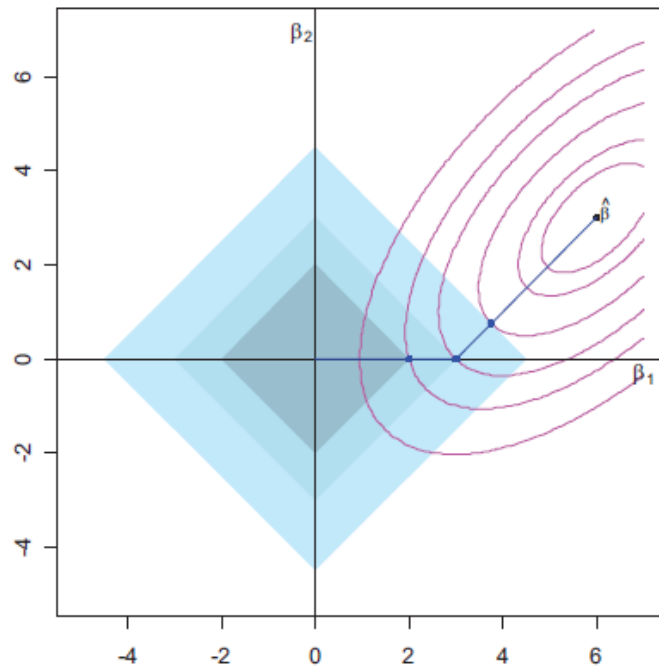


# What to use in practice?

- Supervised learning
  - Linear models: linear regression, logistic regression
    - **Lasso**
    - Ridge regression
    - Elastic net
  - Decision trees: **random forest** and gradient boosting (XGB)
  - **Support vector machines**
  - K-nearest neighbors
  - Neural networks

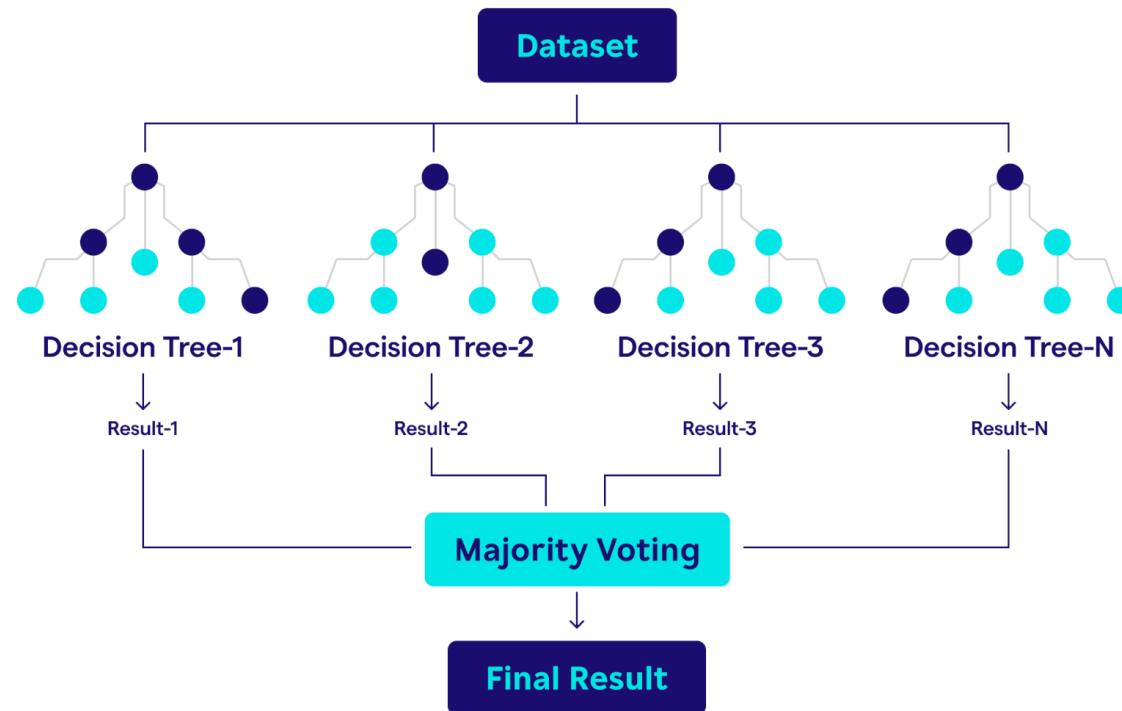
# Lasso: Linear modeling with $l_1$ regularization

- Linear regression's optimization goal:  $\min_{\beta} \frac{1}{2n} \|\mathbf{y} - \mathbf{X}\beta\|_2^2$
- Lasso's optimization goal:  $\min_{\beta} \frac{1}{2n} \|\mathbf{y} - \mathbf{X}\beta\|_2^2 + \lambda \|\beta\|_1$



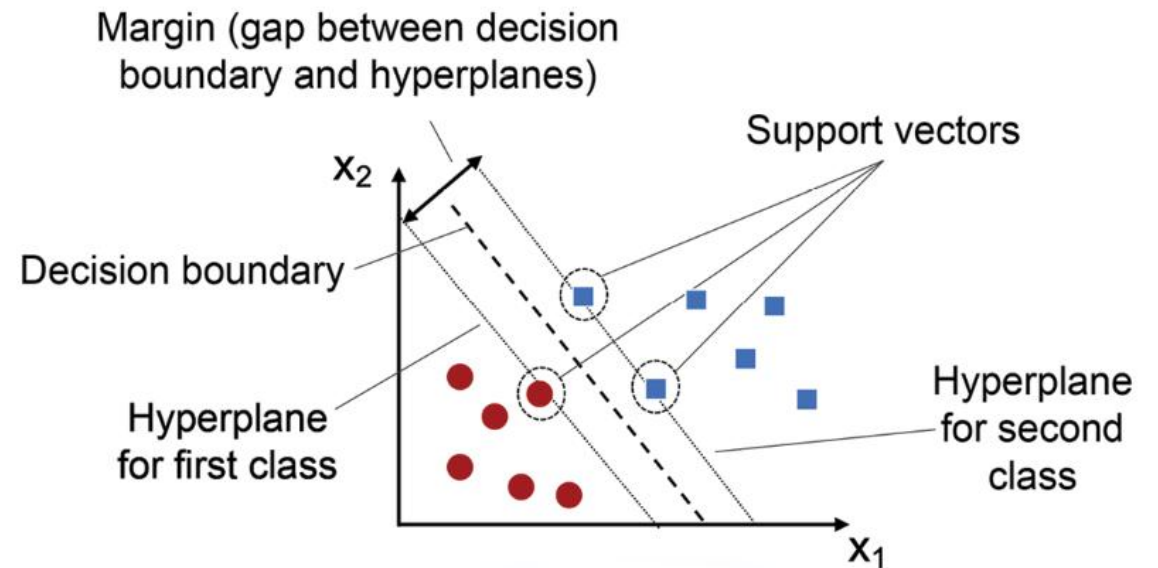
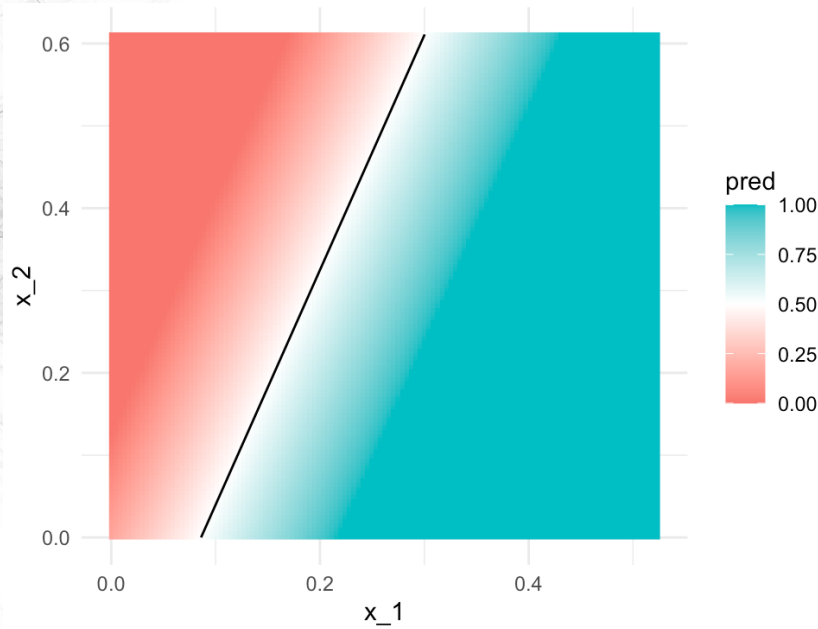
# Random forest: decision tree with bootstrapping

- Each decision tree is constructed based on a subset of data (bootstrapping), and a subset of features.
- Each individual tree overfits, but it is reduced through ensembling many trees.



# Support vector machines: expand the decision boundary

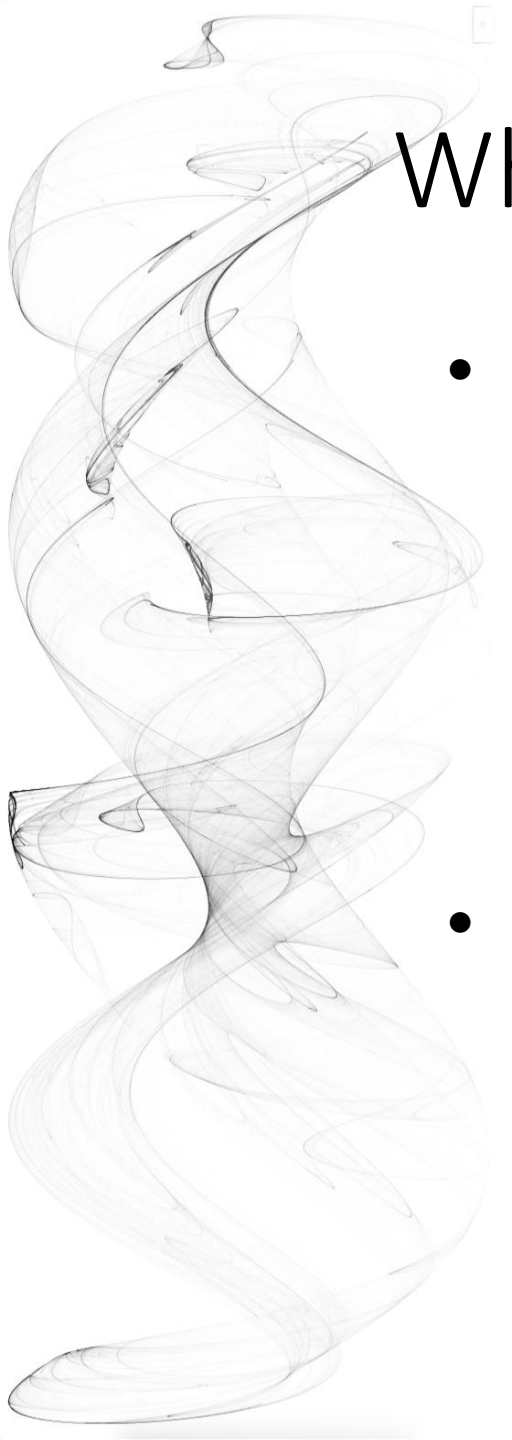
- Derive the decision boundary that has the largest “separation”/“margin”





# Other types of learning

- Unsupervised learning: *e.g., clustering*
- Semi-supervised learning: *e.g., label propagation*
- Self-supervised learning: *e.g., BERT, GPT*
- Weakly supervised learning: *e.g., multiple instance learning*
- Meta-learning: *e.g., few-shot learning*
- Reinforcement learning: *e.g., AlphaGo*
- ...



# What to use in practice?

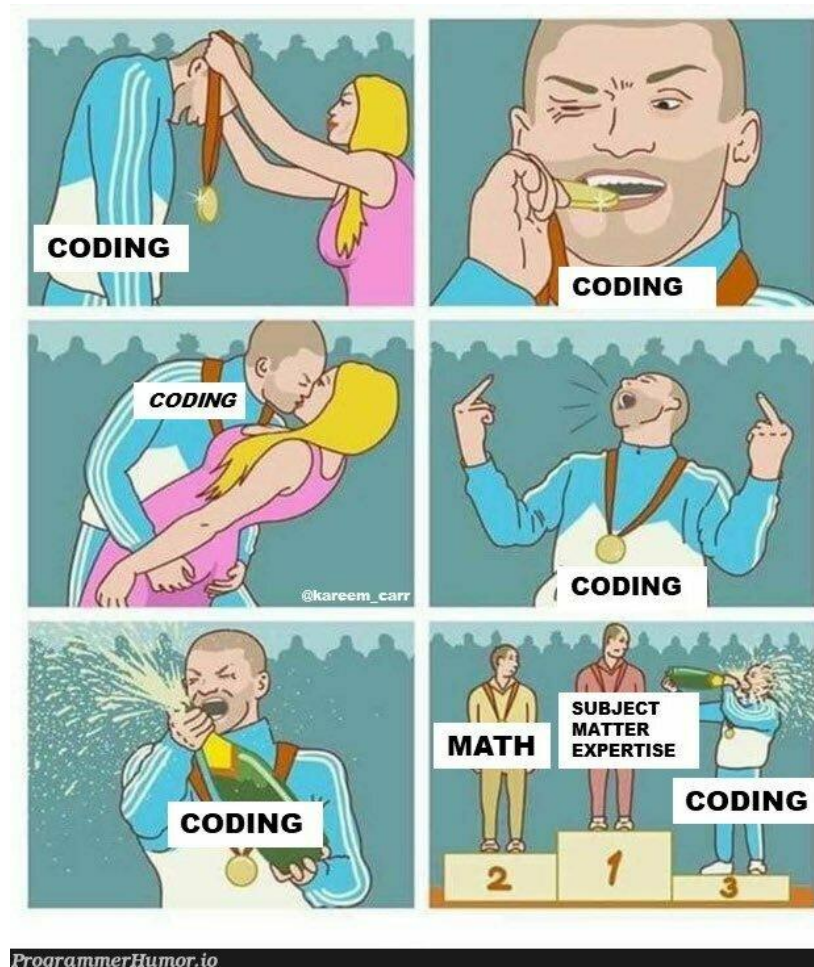
- Python
  - scikit-learn: for evaluations and most machine learning algorithms
  - XGBoost: gradient boosting
  - PyTorch/tensorflow/keras: everything related to neural networks
  - transformers: modern transformer-based architectures
- R
  - caret: for most machine learning algorithms
  - torch for R, and other python packages via reticulate





# In this lecture

- Definition and notation
- Evaluations
- Decision boundary
- What to use in practice



Thank you all for the semester!

Reach out to [zhenqin@hku.hk](mailto:zhenqin@hku.hk) for any questions